Лекція 4

Задача класифікації на основі дерев рішень. Алгоритм CART. Алгоритм ID3. Алгоритм C4.5

Базові концепції методу побудови дерева рішень

Дерева рішень - один з методів автоматичного аналізу даних, основні ідеї якого сягають робіт П. Ховленда (P. Hoveland) і Е. Ханта (Є. Hunt) кінця 50-х років XX ст. У підсумку з'явилася основоположна монографія [Hunt et al., 1966], що дала імпульс розвитку цього напрямку.

Побудова дерев класифікації (рішень) - один з найбільш важливих прийомів, який використовується при проведенні "видобутку даних і розвідувального аналізу" (Data Mining), реалізований як сукупність методів аналітичної обробки великих масивів інформації з метою виявити в них значущі закономірності і / або систематичні зв'язки між змінними, які потім можна застосувати до нових вимірювань.

Дерева рішень являють собою послідовні ієрархічні структури, що складаються з вузлів, які містять правила, тобто логічні конструкції виду "якщо ... то ...". Кінцевими вузлами дерева є "листя", відповідні знайденим рішенням і об'єднуючі деяку кількість об'єктів класифікованої вибірки. Це схоже на те, як положення листа на дереві можна задати, вказавши провідну до нього послідовність гілок, починаючи від кореня і закінчуючи останньою гілочкою, на якій лист зростає.

Є ряд причин, що роблять дерева класифікації більш гнучким засобом, на відміну від традиційних методів аналізу:

• схема одновимірного розгалуження, яка дозволяє вивчати ефект впливу окремих змінних і проводити послідовний аналіз їх вкладу;

• можливість одночасно працювати зі змінними різних типів, виміряних в безперервних і порядкових шкалах, або здійснювати будь-яке монотонне перетворення ознак;

• відсутність попередніх припущень про закони розподілу даних.

Дерева рішень (decisiontrees) призначені для вирішення задач класифікації. Іноді використовують інші назви методу - дерева класифікації, дерева вирішальних правил. Вони формують ієрархічну структуру класифікуючих правил типу «Якщо -То.» (if-then), що має вид дерева. Щоб ухвалити рішення, до якого класу слід віднести деякий об'єкт або ситуацію, вимагається відповісти на питання, що міститься у вузлах цього дерева, починаючи з його кореня. Питання мають вигляд «Значення параметра А більше В?». Якщо відповідь позитивна, здійснюється перехід до правого вузла наступного рівня; потім знову слідує питання, пов'язане з відповідним вузлом і т.д. Наведений приклад ілюструє роботу так званих бінарних дерев рішень, в кожному вузлі яких, галуження проводиться по двох напрямах (тобто на питання, задане у вузлі, є тільки два варіанти відповідей, наприклад «Так» чи «Ні»). Проте, у загальному випадку, відповідей, а отже і віття, що виходить з вузла, може бути більше.

Для побудови дерева, на вхід алгоритму подається деяка навчальна множина, що містить об'єкти (приклади), що характеризуються атрибутами, один з яких вказує на приналежність об'єкту до певного класу. Далі алгоритм намагається виробити загальні критерії для об'єктів одного класу. У тому випадку, якщо навчальна множина містить один або більше прикладів, що відносяться до одного класу, деревом рішень буде лист, який визначатиме даний клас. Якщо ж навчальна множина містить приклади, що відносяться до різних класів, слід розбити його на деякі підмножини. Для цього вибирається один з атрибутів, що має два і більше відмінних один від одного значень. Після розбиття кожна підмножина буде містити всі приклади, що мають одне із значень для вибраного атрибута. Ця процедура буде рекурсивно продовжуватися до тих пір, поки кінцева множина не буде складатися з прикладів, що відносяться до одного і того ж класу. Серед незаперечних позитивних сторін дерев рішень можливо виділити швидкий процес навчання, знаходження правил в областях, де експерту важко формалізувати свої знання, інтуїтивно зрозумілу класифікаційну модель, високу точність класифікації, побудову непараметричних моделей. Через ці та багато інших причин, дерева рішень є важливим інструментом в роботі кожного фахівця, що займається

аналізом даних. На жаль, до цього часу не досліджено ще багато потенційних областей застосування дерев рішень.

Якість побудованого дерева після навчання можна оцінити за декількома параметрами. По-перше, це число розпізнаних прикладів в навчальному і тестовому наборах даних. Чим воно вище, тим якісніше побудоване дерево. По-друге, це кількість вузлів в дереві. При дуже великому їх числі дерево стає важким для сприйняття. Це також означає дуже слабку залежність вихідного поля від вхідних полів. Кожне правило характеризується підтримкою та достовірністю.

Підтримка – це загальна кількість прикладів класифікованих даним вузлом дерева. **Достовірність** – це кількість правильно класифікованих даним вузлом прикладів. Дерево рішень будується за визначеним алгоритмом.

Область застосування дерев рішень дуже широка. Для спрощення засвоєння матеріалу є сенс усі задачі, що вирішуються цим методом поєднати у три класи.

1. Опис даних. Дерева рішень дозволяють зберігати інформацію про дані в компактній формі. Замість громіздких масивів даних їх можна зберегти у вигляді дерева рішень, яке містить точний опис об'єктів.

2. Класифікація. Дерева рішень відмінно справляються із задачами класифікації, тобто віднесенням об'єктів до одного з наперед відомих класів.

3. Регресія. Якщо цільова змінна має неперервні значення, дерева рішень дозволяють встановити залежність цільової змінної від незалежних (вхідних) змінних. Наприклад, до цього класу відносяться задачі чисельного прогнозування (прогнози значень цільової змінної).

Для побудови дерева рішень готується навчальна вибірка. При цьому вихідне поле дерева рішень може бути тільки дискретним. Для полів, що подаються на входи і вихід дерева рішень, також задається нормалізація. Можливо задати або лінійну нормалізацію, або нормалізацію унікальними значеннями.

Для прийняття рішення за допомогою "дерева рішень" необхідно виконати такі кроки:

1) оцінити стан ринку вектором чинників X = (x1, x2, x3, x4) (цей крок виконується користувачем системи);

2) визначити клас зростання прибутку шляхом руху вектора X = (x1, x2, x3, x4) по дереву рішень з верхніх рівнів до нижніх (цей крок виконується системою).

У методиці використовується ієрархічна структурна схема. Для її побудови прийняті відповідні позначення елементів (подій) і логічних операцій.

Ключовою перевагою "дерева рішень" перед іншими методами є можливість пов'язати ціль з діями, що підлягають реалізації. При побудові багаторівневого "дерева рішень" досягнення мети кожного з рівнів моделі забезпечується комплексом заходів попереднього рівня. Кожен рівень "дерева рішень" повинен займати певне місце в ієрархічній послідовності, складеної на основі дотримання причинно-наслідкових зв'язків.

Етапи побудови "дерева рішень"

Дерево рішень - це графічне зображення послідовності рішень і станів середовища з вказівкою відповідних ймовірностей та виграшів для будь-яких комбінацій альтернатив і станів середовища.

Побудова "дерева рішень" виконується "зверху вниз" - від задач більш складних, більш важливих - до менш складних завдань, що вимагає менше часу (коштів, сил, ресурсів).

На схемі "дерева рішень" найвище положення займає кінцева мета вирішення проблеми (кінцевий результат). Чим складніше можна вирішити завдання, тим більше має бути число рівнів розгляду проблеми і тим більше число завдань, що вирішуються на кожному рівні.

Для кожного "дерева рішень" будується матриця. Часто вводяться коефіцієнти взаємної корисності рішень, одержувані опитуванням експертів. Вони показують вплив ступеня важливості одних рішень на інші.

Застосування методу "дерева рішень" дозволяє:

- визначати шляхи досягнення мети з виконанням кількісної оцінки складності завдань, які виникають та здійснити оцінку труднощі здійснення того чи іншого завдання;
- поліпшити якість рішень в умовах невизначеності.

Процес прийняття управлінських рішень за допомогою дерева рішень у загальному випадку припускає виконання п'яти етапів:

Етап 1. Формулювання завдання.

Насамперед необхідно відкинути всі фактори, що не стосуються проблеми, а серед безлічі тих, що залишилися, виділити суттєві і несуттєві. Це дозволить провести опис завдання щодо представлення управлінського рішення у формі, що піддається аналізу. Повинні бути виконані такі основні процедури:

- визначення можливостей збору інформації для експериментування і реальних дій;
- складання переліку подій, що з певною імовірністю можуть відбутися;
- встановлення часового порядку розміщення подій, у наслідках яких міститься корисна і доступна інформація, і тих послідовних дій, які можна розпочати.

Етап 2. Побудова "дерева рішень".

Етап 3. Оцінка ймовірностей станів середовища, тобто зіставлення шансів виникнення кожної конкретної події. Слід зазначити, що вказані ймовірності визначаються або на підставі наявної статистики, або експертним шляхом.

Етап 4. Встановлення виграшів (чи програшів, як виграшів зі знаком мінус) для кожної можливої комбінації альтернатив (дій) і станів середовища.

Етап 5. Вирішення завдання.

Правила побудови «дерева рішень»

При побудові дерев рішень особлива увага приділяється таким питанням: вибір критерію атрибута, за яким буде проведено розбиття, зупинка навчання та відсікання гілок.

Правило розбиття – яким чином потрібно обирати ознаку?

Для побудови дерева на кожному внутрішньому вузлі необхідно знайти таку умову (перевірку), яка б розбивала множину, асоційовану із цим вузлом, на підмножини. В якості такої перевірки можна вибрати один із атрибутів – так званий ***атрибут розщеплення***. Загальне правило для вибору атрибута можна сформулювати так: обраний атрибут повинен розбивати множину так, щоб одержані підмножини складалися з об'єктів одного класу або були максимально наближені до цього, тобто кількість об'єктів з інших класів («домішок») у кожній з одержаних підмножин повинна бути якомога меншою. Серед безлічі розроблених критеріїв для вибору найкращого атрибута виділимо два:

1. Теоретико-інформаційний критерій – алгоритм C4.5, що є удосконаленою версією алгоритму ID3 (Iterative Dichotomizer) – використовує теоретико-інформаційний підхід.
2. Статистичний критерій – алгоритм CART, що використовує так званий індекс Gini (в честь італійського економіста Corrado Gini) для оцінювання «відстані» між розподілами класів.

Правило зупинки. Розбивати вузол далі, чи визначати його як листок?

На додачу до основного методу побудови дерев рішень були запропоновані такі правила:

1. Використання статистичних методів для оцінювання доцільності подальшого розбиття – так звана «рання зупинка» (prepruning). «Рання зупинка» процесу побудови дозволяє зекономити час навчання, проте реалізує менш точні класифікаційні моделі. Тому використання «ранньої зупинки» в більшості випадків є небажаним.
2. Можна задати глибину дерева. Тобто подальше розбиття не повинне привести до дерева з глибиною, що перевищує задане значення.
3. Можна задати мінімальну кількість об’єктів у вузлі. Тобто розбиття повинне бути нетривіальним, щоб вузли, одержані в результаті розбиття, містили не менше заданої кількості об’єктів.

Цей список евристичних правил можна продовжити, але нині не існує домінуючого правила. До кожного конкретного випадку потрібно підходити індивідуально.

Правило відсікання. Яким чином потрібно відсікати гілки дерева?

Дуже часто алгоритми побудови дерев рішень видають складні, «переповнені даними» дерева, що мають багато вузлів і гілок. Такі сильно розгалужені дерева дуже важкі для розуміння. Крім того, сильно «гіллясте» дерево з багатьма вузлами розбиває навчальну множину на велику кількість маленьких підмножин. Проте цінність правила, що містить лише 2-3 об'єкти є вкрай низькою, тому таке правило не можна використати для аналізу даних. Набагато краще мати дерево, що містить невелику кількість вузлів з великою кількістю об'єктів з навчальної вибірки. Виникає питання: чи можна побудувати всі можливі варіанти дерев для однієї навчальної множини, а тоді вибрати дерево з найменшою глибиною? На жаль, це завдання не має ефективних методів вирішення.

Для розв'язання описаної вище проблеми часто використовується так зване відсікання гілок (pruning).

Нехай точність (розпізнавання) дерева рішень – це відношення правильно класифікованих об'єктів при навчанні до загальної кількості об'єктів з навчальної множини. Тоді помилка – це відношення кількості неправильно класифікованих об'єктів до загальної кількості об'єктів у навчальній множині.

Якщо відомий спосіб оцінювання помилки дерева, гілок та листків, то можна використати такі прості правила:

1. побудувати дерево;
2. відсікти або замінити під деревом ті гілки, відсічення яких не приведе до зростання помилки.

На відміну від процесу побудови дерева, відсікання його гілок відбувається знизу вверх, рухаючись від листків дерева, позначаючи вузли як листки, або заміняючи їх піддеревом. Процедура відсікання є більш популярною, ніж використання правил зупинки. Дерева, одержані в результаті відсікання називають ***усіченими деревами***.

Хоча відсікання не є панацеєю, проте у більшості практичних завдань дає гарні результати, що дозволяє говорити про правомірність використання подібної методики.

Алгоритм CART

Алгоритм CART (Classification and Regression Tree), як бачимо з назви, призначений для вирішення задач класифікації об'єктів і побудови регресійної моделі. Він розроблений в 1974-1984 роках чотирма професорами статистики - Leo Breiman (Berkeley), Jerry Friedman (Stanford), Charles Stone (Berkeley) і Richard Olshen (Stanford).

Атрибути набору даних можуть мати як дискретне, так і числове значення. Алгоритм CART призначений для побудови бінарного дерева рішень. Бінарні дерева також називають двійковими.

Інші особливості алгоритму CART:

• функція оцінки якості розбиття;
• механізм відсікання дерева;
• алгоритм обробки пропущених значень;
• побудова дерев регресії.

Кожен вузол бінарного дерева при розбитті має тільки двох нащадків, званих дочірніми гілками. Подальший поділ гілки залежить від того, чи багато вихідних даних описує дана гілка. На кожному кроці побудови дерева, правило, сформоване у вузлі, ділить задану кількість прикладів на дві частини. Права частина (гілка right) - це та частина множини, в якій зазвичай виконується правило; ліва (гілка left)- та, для якої правило не виконується.

Функцією оцінки якості розбиття, яка використовується для вибору оптимального правила є індекс Gini. Відзначимо, що дана функція заснована на ідеї зменшення невизначеності в вузлі. Припустимо, є вузол, і він розбитий на два класи. Максимальна невизначеність у вузлі буде досягнута при розбитті його на дві підмножини по 50 прикладів, а максимальна визначеність - при розбитті на 100 і 0 прикладів.

Нагадаємо, що алгоритм CART працює з числовими і категоріальними атрибутами. У кожному вузлі розбиття може відбуватись тільки по одному атрибуту. Якщо атрибут є числовим, то у внутрішньому вузлі формується правило виду $x_i <= c$, Значення c в більшості випадків вибирається як середнє арифметичне двох сусідніх впорядкованих значень змінної x_i навчального набору даних. Якщо ж атрибут відноситься до категоріального типу, то у

внутрішньому вузлі формується правило x_i $V(x_i)$, де $V(x_i)$ - деяка не порожня підмножина множини значень змінної x_i в навчальному наборі даних.

Механізм відсікання. Цим механізмом, що має назву minimal cost-complexity tree pruning, алгоритм CART принципово відрізняється від інших алгоритмів конструювання дерев рішень. У розглянутому алгоритмі відсікання - це певний компроміс між отриманням дерева "підходящого розміру" і отриманням найбільш точної оцінки класифікації. Метод полягає в отриманні послідовності зменшуваних дерев, але дерева розглядаються не всі, а тільки "кращі представники".

Перехресна перевірка (V-fold cross-validation) є найбільш складною і одночасно оригінальною частиною алгоритму CART. Вона являє собою шлях вибору остаточного дерева, за умови, що набір даних має невеликий об'єм або ж записи набору даних настільки специфічні, що розділити набір на навчальну та тестову вибірку не можливо.

Отже, основні характеристики алгоритму CART: бінарне розщеплення, критерій розщеплення - індекс Gini, алгоритми minimal cost-complexity tree pruning і V-fold cross-validation, принцип "виростити дерево, а потім скоротити", висока швидкість побудови, обробка пропущених значень

Розглянемо задачу з двома класами і вузлом, який містить 50 прикладів одного класу. Нехай вузол має максимальну «забрудненість». Якщо буде знайдено розбиття, яке розбиває дані, наприклад, на дві підгрупи (40:5 прикладів в одній і 10:45 в іншій), то інтуїтивно «забрудненість» буде зменшено. Вона повністю зникне, коли буде знайдено розбиття, яке створить підгрупи 50:0 та 0:50. В даному алгоритмі ідея «забрудненості» формалізована в індексі Gini. Якщо набір даних T містить дані n класів, тоді даний індекс обраховується, як

$$Gini(T) = 1 - \sum_{i=1}^{n} p_i^2 , \qquad (4.1)$$

де параметр p_i – ймовірність класу i в T.

Якщо набір T розбивається на дві частини T_1 і T_2 з числом прикладів в кожному N_1 і N_2 відповідно, то показник якості розбиття буде

$$Gini_{split}(T) = \frac{N_1}{N} Gini(T_1) + \frac{N_2}{N} Gini(T_2) , \qquad (4.2)$$

Найкращим вважається те розбиття, для якого $Gini_{split}(T)$ найменше.

Процес побудови регресійних дерев рішень аналогічний до побудови класифікаційних, але замість міток класів в листі будуть розташовуватися числові значення. Фактично при цьому реалізується кусочно-постійна функція вхідних змінних.

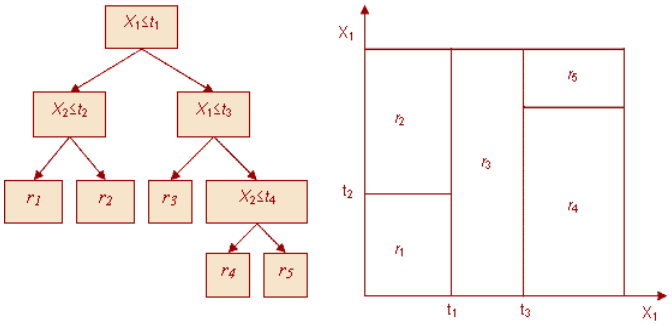


Рис. 4.1. Дерево рішень побудоване за допомогою алгоритму CART.

Переваги алгоритму

- Даний алгоритм є непараметричним, це означає, що для його застосування немає необхідності розраховувати різні параметри ймовірнісного розподілу.
- Для застосування алгоритму CART немає необхідності заздалегідь вибирати змінні, які братимуть участь в аналізі: змінні відбираються безпосередньо під час проведення аналізу на підставі значення індексу Gini.
- CART легко бореться з викидами: механізм «розбиття» закладений в алгоритмі просто поміщає «викиди» в окремий вузол, що дозволяє очистити наявні дані від шумів.

- Для застосування цього алгоритму не треба брати до уваги ніяких припущень або припущень перед проведенням аналізу.
- Великою перевагою є швидкість роботи алгоритму.

Недоліки алгоритму

- Дерева рішень, запропоновані алгоритмом, не є стабільними: результат, отриманий на одній вибірці, буває не відтворюваним на іншій (дерево може збільшуватися, зменшуватися і т.д.)
- У разі, коли необхідно побудувати дерево з більш складною структурою, краще використовувати інші алгоритми, так як CART може неправильно ідентифікувати структуру даних.

2. Приклад роботи алгоритму

Дано навчальну вибірку, що містить різні атрибути, які визначають, чи буде проведено матч.

Таблиця 4.1

День	Погода	Темп-ра	Вологість	Вітер	Рішення
1	Сонячна	Висока	Висока	Слабкий	Ні
2	Сонячна	Висока	Висока	Сильний	Ні
3	Хмарна	Висока	Висока	Слабкий	Так
4	Дощова	Помірна	Висока	Слабкий	Так
5	Дощова	Низька	Нормальна	Слабкий	Так
6	Дощова	Низька	Нормальна	Сильний	Ні
7	Хмарна	Низька	Нормальна	Сильний	Так
8	Сонячна	Помірна	Висока	Слабкий	Ні
9	Сонячна	Низька	Нормальна	Слабкий	Так
10	Дощова	Помірна	Нормальна	Слабкий	Так
11	Сонячна	Помірна	Нормальна	Сильний	Так
12	Хмарна	Помірна	Висока	Сильний	Так
13	Хмарна	Висока	Нормальна	Слабкий	Так
14	Дощова	Помірна	Висока	Сильний	Ні

Порахуємо для кожного атрибуту (погода, температура, вологість, вітер) індекс Gini.

Погода

Таблиця 4.2

Погода	Так	Ні	Кількість
Сонячна	2	3	5
Хмарна	4	0	4
Дощова	3	2	5

$$Gini(T) = 1 - \sum_{i=1}^{n} p_i^2$$

Gini(Погода=Сонячна) = 1 – $(2/5)^2$ – $(3/5)^2$ = 0.48
Gini(Погода =Хмарна) = 1 – $(4/4)^2$ – $(0/4)^2$ = 0
Gini(Погода =Дощова) = 1 – $(3/5)^2$ – $(2/5)^2$ = 0.48

Порахуємо суму індексів для погоди:

Gini(Погода) = (5/14) x 0.48 + (4/14) x 0+ (5/14) x 0.48 = 0.342

Температура

Таблиця 4.3

Темп-ра	Так	Ні	Кількість
Висока	2	2	4
Холодна	3	1	4
Помірна	4	2	6

Gini(Темп=Висока) = 1 – $(2/4)^2$ – $(2/4)^2$ = 0.5
Gini(Темп=Холодна) = 1 – $(3/4)^2$ – $(1/4)^2$ = 0.375
Gini(Темп=Помірна) = 1 – $(4/6)^2$ – $(2/6)^2$ = 0.445

Порахуємо суму індексів для темп:

Gini(Temp) = (4/14) x 0.5 + (4/14) x 0.375 + (6/14) x 0.445 = 0.439

Вологість

Таблиця 4.4

Вологість	Так	Ні	Кількість
Висока	3	4	7
Нормальна	6	1	7

Gini(Вологість=Висока) = 1 – $(3/7)^2$ – $(4/7)^2$ = 0.489
Gini(Вологість=Нормальна) = 1 – $(6/7)^2$ – $(1/7)^2$ = 0.244

Порахуємо суму індексів для вологості:

Gini(Вологість) = (7/14) x 0.489 + (7/14) x 0.244 = 0.367

Вітер

Таблиця 4.5

Вітер	Так	Ні	Кількість
Слабкий	6	2	8
Сильний	3	3	6

Gini(Вітер=Слабкий) = $1 – (6/8)^2 – (2/8)^2$ 2 = 0.375
Gini(Вітер=Сильний) = $1 – (3/6)^2 – (3/6)^2$ = 0.5

Порахуємо суму індексів для вітру:

Gini(Вітер) = (8/14) x 0.375 + (6/14) x 0.5 = 0.428
Базуючись на індексах Gini визначено, що атрибут «погода» буде коренем дерева.

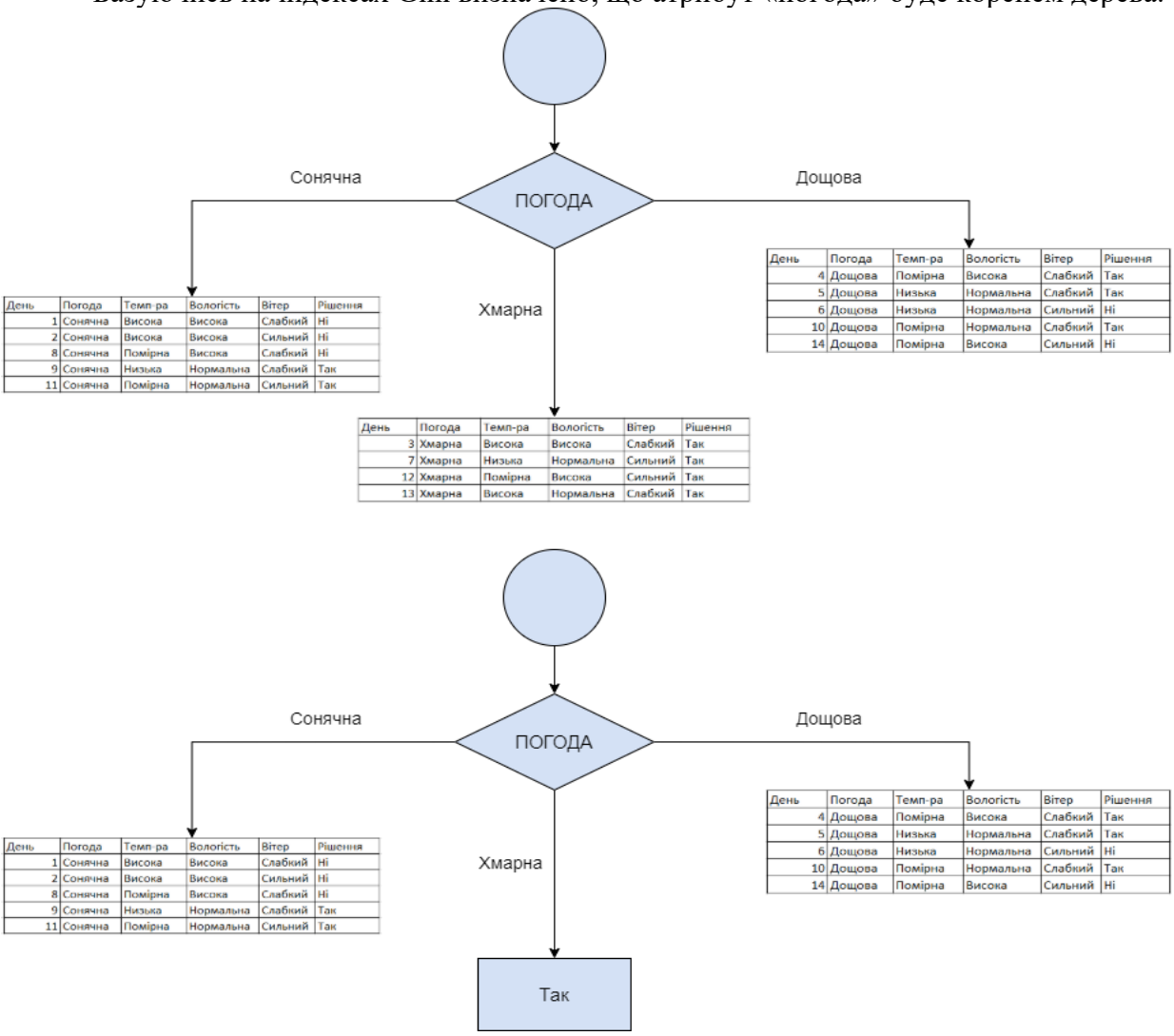
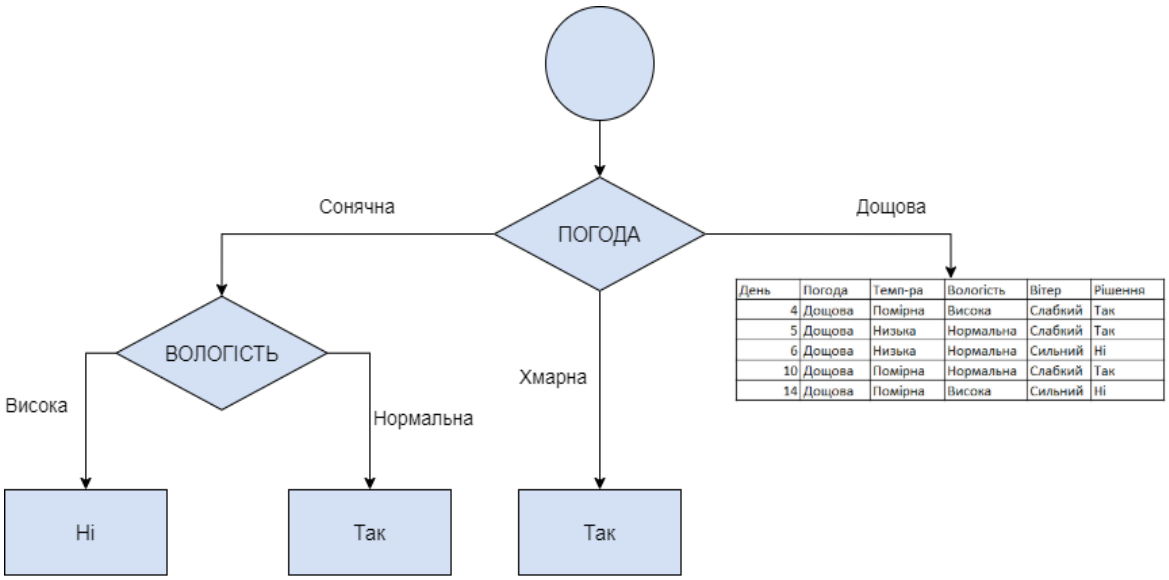


Рис. 4.2. Проміжний варіант дерева рішень

Так як при хмарній погоді отримуємо підмножину, яка містить об’єкти лише одного класу “Так”, тому одразу визначаємо, що матч відбудеться.

Аналогічні дії з розрахунком індексів Gini було проведено для атрибутів погода «сонячна», вологість, температура, вітер.

Індекс для атрибуту «вологість» виявився найменшим, тому продовжуємо роботу з даним атрибутом. При високій вологості отримаємо підмножину, яка містить лише елементи класу «Ні», а при нормальній – «Так». Розбиття даної вітки завершено.



День	Погода	Темп-ра	Вологість	Вітер	Рішення
4	Дощова	Помірна	Висока	Слабкий	Так
5	Дощова	Низька	Нормальна	Слабкий	Так
6	Дощова	Низька	Нормальна	Сильний	Ні
10	Дощова	Помірна	Нормальна	Слабкий	Так
14	Дощова	Помірна	Висока	Сильний	Ні

Рис. 4.3. Проміжний варіант дерева рішень

Аналогічні дії з розрахунком індексів Gini було проведено для атрибутів погода «дощова», вологість, температура, вітер.

Індекс для атрибуту «вітер» виявився найменшим, тому продовжуємо роботу з даним атрибутом. При сильному вітрі отримаємо підмножину, яка містить лише елементи класу «Ні», а при слабкому – «Так».

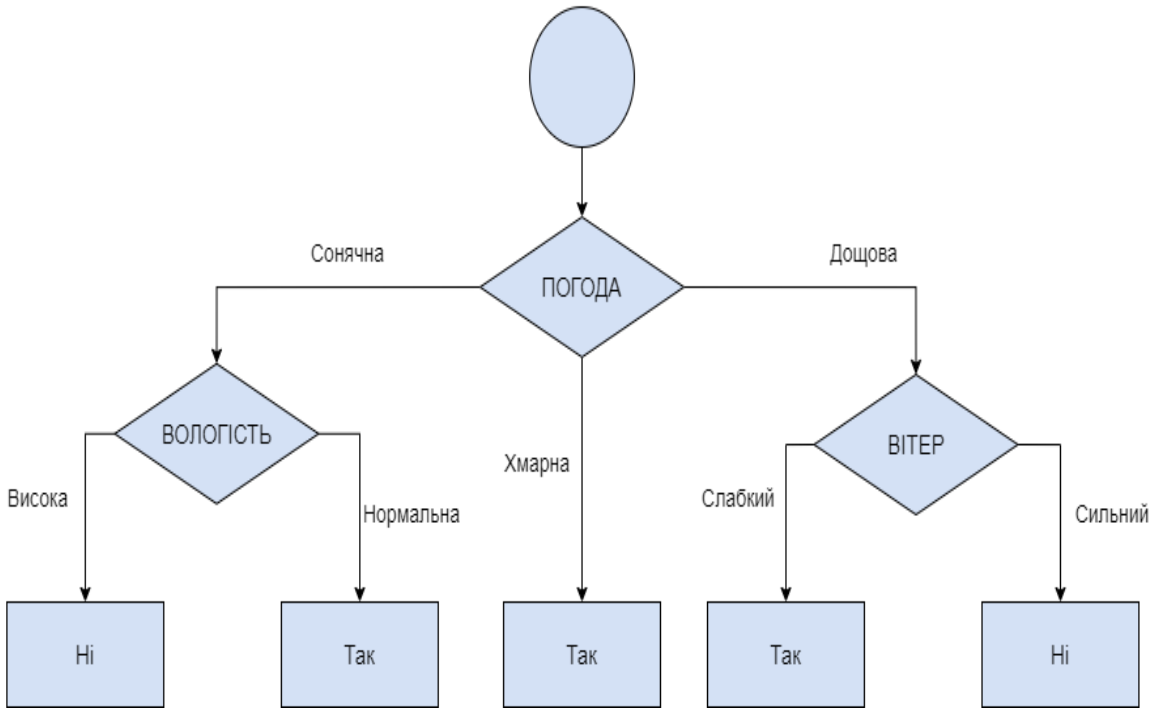


Рис. 4.4. Остаточне дерево рішень

Побудову дерева рішень завершено.

Алгоритм ID3

ID3 - це алгоритм, який використовується для генерації дерев рішень у машинному навчанні з деякого набору даних. ID3 є попередником алгоритму C4.5.

Властивості алгоритму

Алгоритм ID3 не гарантує знаходження оптимального рішення. Він використовує жадібну стратегію, обираючи локальний кращий атрибут для розбиття множини даних на кожній ітерації. Оптимальність алгоритму може бути покращена шляхом використання пошуку з вертанням під час пошуку оптимального дерева рішень, але це може призвести до

погіршення швидкості роботи.

ID3 може перенавчатися. Для уникнення перенавчання варто надавати перевагу меншим деревам рішень. Даний алгоритм зазвичай будує невеликі дерева, але він не завжди дає найменше дерево рішень.

ID3 важче використовувати на неперервному наборі даних. Якщо значення будь-якого атрибута неперервні, то існує багато місць для розбиття даних за цим атрибутом і пошук найкращого значення для розбиття може зайняти багато часу.

Основні кроки алгоритму:

- Обчислити ентропію загальної множини S;
- Обчислити інформаційні прирости кожного з атрибутів множини S;
- Коренем дерева стане атрибут з найбільшим інформаційним приростом;
- Повторити попередні кроки для всіх підмножин, використовуючи атрибути, що залишились.

Формули, які використовуються при реалізації даного алгоритму:

$$H(S) = -\sum_{i=0}^{i=n} p(x_i)\log_2 p(x_i) - \text{ентропія множини } S \tag{4.3}$$

$$IG(S,A) = H(S) - H(S|A) - \text{інформаційний приріст множини } S \text{ при розділенні атрибутом A} \tag{4.4}$$

$$H(S|A) = \sum_{t \in T} p(t)H(t) - \text{ентропія множини } S \text{ після розбиття за атрибутом A} \tag{4.5}$$

Переваги алгоритму

- Легко інтерпретується (через простоту моделі, можна легко відобразити дерево і простежити за всіма вузлами дерева).
- Простота класифікації.

Недоліки алгоритму

- Алгоритм не працює з неперервними (числовими) атрибутами.
- Алгоритм не передбачає можливості роботи з пропусками в даних.

1. Приклад роботи алгоритму

Навчальна вибірка для побудови дерева рішень

Об’єкт дослідження – безпека купання людини у водоймах

Таблиця 4.6

№ людини	Стан здоров'я людини	Рівень забрудненості водойми	Температура повітря	Природні явища	Чи безпечно купатись
1	Задовільний	Середній	Висока	Гроза	Ні
2	Незадовільний	Низький	Середня	Відсутні	Ні

3	Задовільний	Середній	Висока	Відсутні	Так
4	Задовільний	Високий	Середня	Відсутні	Ні
5	Задовільний	Низький	Дуже висока	Відсутні	Ні
6	Незадовільний	Низький	Середня	Град	Ні
7	Задовільний	Низький	Висока	Відсутні	Так
8	Задовільний	Середній	Низька	Відсутні	Ні
9	Задовільний	Середній	Низька	Дощ	Ні
10	Задовільний	Низький	Висока	Гроза	Ні
11	Задовільний	Низький	Середня	Відсутні	Так
12	Задовільний	Середній	Дуже висока	Відсутні	Ні
13	Задовільний	Середній	Середня	Град	Ні
14	Незадовільний	Низький	Висока	Відсутні	Ні
15	Незадовільний	Високий	Дуже висока	Дощ	Ні
16	Задовільний	Середній	Середній	Відсутні	Так
17	Задовільний	Високий	Низька	Дощ	Ні

1. Обрахунок ентропії всієї вибірки

S (+4; -13)

$$H(S) = -(\frac{4}{17} \cdot \log_2 \frac{4}{17} + \frac{13}{17} \cdot \log_2 \frac{13}{17}) = 0{,}787$$

2. Обрахунок інформаційного приросту для атрибуту «Стан здоров'я людини» {«Задовільний», «Незадовільний»}

$S_{\text{задовільний}}$ (+9; -4)

$$H(S_{\text{задовільний}}) = -\left(\frac{9}{13} \cdot \log_2 \frac{9}{13} + \frac{4}{13} \cdot \log_2 \frac{4}{13}\right) = 0{,}89$$

$S_{\text{незадовільний}}$ (+0; -4)

$$H(S_{\text{незадовільний}}) = -\left(\frac{0}{4} \cdot \log_2 \frac{0}{4} + \frac{4}{4} \cdot \log_2 \frac{4}{4}\right) = 0$$

$$IG(S, A) = H(S) - H(S|A) = 0{,}787 - \left(\frac{13}{17} \cdot 0{,}89 + \frac{4}{17} \cdot 0\right) = 0.106$$

Обрахунок інформаційного приросту для атрибуту «Рівень забрудненості водойми» {«Низький», «Середній», «Високий»}

$S_{\text{низький}}$ (+2; -5)

$$H(S_{\text{низький}}) = -\left(\frac{2}{7}\cdot\log_2\frac{2}{7}+\frac{5}{7}\cdot\log_2\frac{5}{7}\right) = 0{,}86$$

$S_{\text{середній}}$ (+1; -6)

$$H\left(S_{\text{середній}}\right) = -\left(\frac{1}{7}\cdot\log_2\frac{1}{7}+\frac{6}{7}\cdot\log_2\frac{6}{7}\right) = 0{,}59$$

$S_{\text{високий}}$ (+0; -3)

$$H(S_{\text{високий}}) = -\left(\frac{0}{3}\cdot\log_2\frac{0}{3}+\frac{3}{3}\cdot\log_2\frac{3}{3}\right) = 0$$

$$IG(S,A) = H(S) - H(S|A) = 0{,}787 - \left(\frac{7}{17}\cdot 0{,}86 + \frac{7}{17}\cdot 0{,}59 + \frac{3}{17}\cdot 0\right) = 0{,}18$$

Обрахунок інформаційного приросту для атрибуту «Температура повітря» {«Низька», «Середня», «Висока», «Дуже висока»}

$S_{\text{низька}}$ (+0; -3)

$$H(S_{\text{низька}}) = -\left(\frac{0}{3}\cdot\log_2\frac{0}{3}+\frac{3}{3}\cdot\log_2\frac{3}{3}\right) = 0$$

$S_{\text{середня}}$ (+2; -4)

$$H\left(S_{\text{середня}}\right) = -\left(\frac{2}{6}\cdot\log_2\frac{2}{6}+\frac{4}{6}\cdot\log_2\frac{4}{6}\right) = 0{,}92$$

$S_{\text{висока}}$ (+2; -3)

$$H(S_{\text{висока}}) = -\left(\frac{2}{5}\cdot\log_2\frac{2}{5}+\frac{3}{5}\cdot\log_2\frac{3}{5}\right) = 0{,}971$$

$S_{\text{дуже висока}}$ (+0; -3)

$$H\left(S_{\text{дуже висока}}\right) = -\left(\frac{0}{3}\cdot\log_2\frac{0}{3}+\frac{3}{3}\cdot\log_2\frac{3}{3}\right) = 0$$

$$IG(S,A) = H(S) - H(S|A) = 0{,}787 - \left(\frac{3}{17}\cdot 0 + \frac{6}{17}\cdot 0{,}92 + \frac{5}{17}\cdot 0{,}971 + \frac{3}{17}\cdot 0\right) = 0{,}177$$

- Сортування інформаційних приростів кожного з атрибутів за спаданням
  - $IG(S,A) = 0{,}216$ - інформаційний приріст атрибуту «Природні явища»
  - $IG(S,A) = 0{,}18$ - інформаційний приріст атрибуту «Рівень забрудненості водойми»
  - $IG(S,A) = 0{,}177$ - інформаційний приріст атрибуту «Температура повітря»
  - $IG(S,A) = 0{,}106$ - інформаційний приріст атрибуту «Стан здоров'я людини»
- Новий корінь дерева – атрибут «Природні явища»

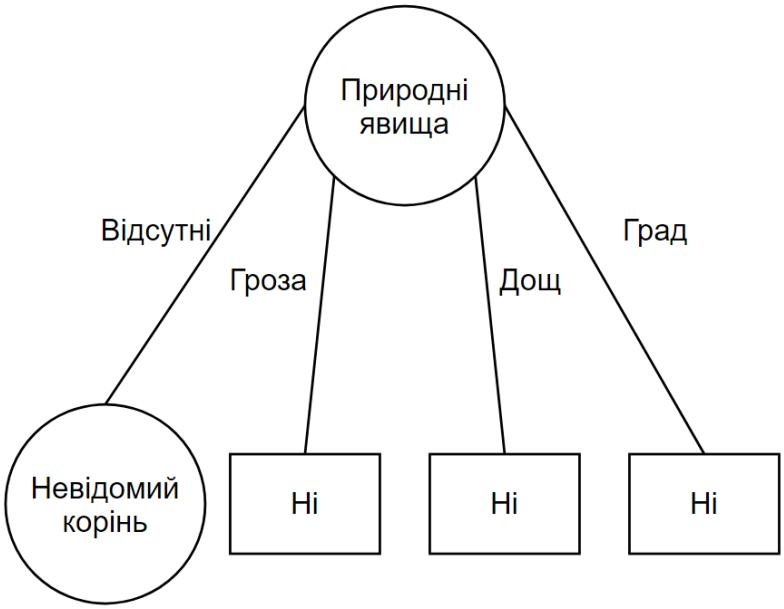


Рис. 4.5. Проміжний варіант дерева рішень

Отримаємо нову підмножину

Таблиця 4.7

№ людини	Стан здоров'я людини	Рівень забрудненості водойми	Температура повітря	Чи безпечно купатись
2	Незадовільний	Низький	Середня	Ні
3	Задовільний	Середній	Висока	Так
4	Задовільний	Високий	Середня	Ні
5	Задовільний	Низький	Дуже висока	Ні
7	Задовільний	Низький	Висока	Так
8	Задовільний	Середній	Низька	Ні
11	Задовільний	Низький	Середня	Так
12	Задовільний	Середній	Дуже висока	Ні
14	Незадовільний	Низький	Висока	Ні
16	Задовільний	Середній	Середня	Так

На основі цієї підмножини аналогічно здійснюємо пошук невідомого кореня дерева рішень.

- Сортування інформаційних приростів кожного з атрибутів за спаданням
  - $IG(S,A) = 0{,}294$ - інформаційний приріст атрибуту «Температура повітря»
  - $IG(S,A) = 0{,}17$ - інформаційний приріст атрибуту «Стан здоров'я людини»
  - $IG(S,A) = 0{,}09$ - інформаційний приріст атрибуту «Рівень забрудненості водойми»
- Новий корінь дерева – атрибут «Температура повітря»

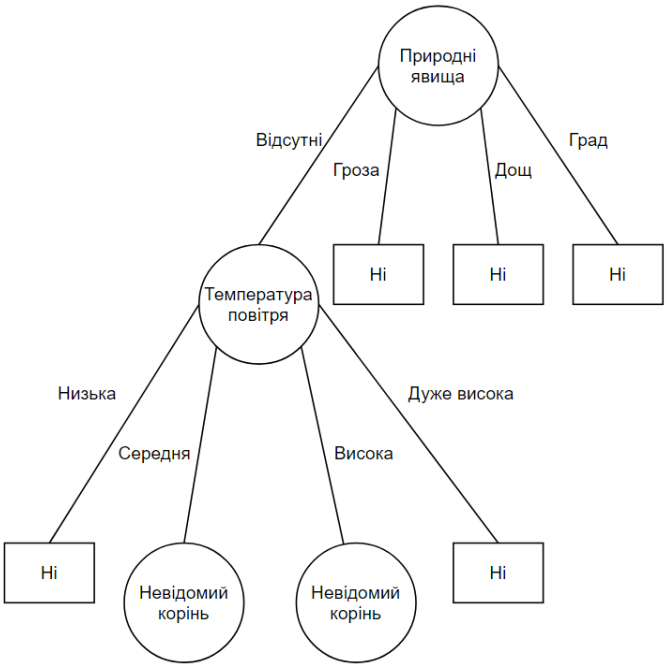


Рис. 4.6. Проміжний варіант дерева рішень

Отримаємо нову підмножину

Таблиця 4.8

№ людини	*Стан здоров'я людини*	*Рівень забрудненості водойми*	*Чи безпечно купатись*
2	*Незадовільний*	*Низький*	*Ні*
4	*Задовільний*	*Високий*	*Ні*
11	*Задовільний*	*Низький*	*Так*
16	*Задовільний*	*Середній*	*Так*

На основі цієї підмножини аналогічно здійснюємо пошук невідомого кореня дерева рішень.

- Сортування інформаційних приростів кожного з атрибутів за спаданням
  - $IG(S,A) = 0,5$ - інформаційний приріст атрибуту «Рівень забрудненості водойми»
  - $IG(S,A) = 0,2725$ - інформаційний приріст атрибуту «Стан здоров'я людини»
- Новий корінь дерева – атрибут «Рівень забрудненості водойми»

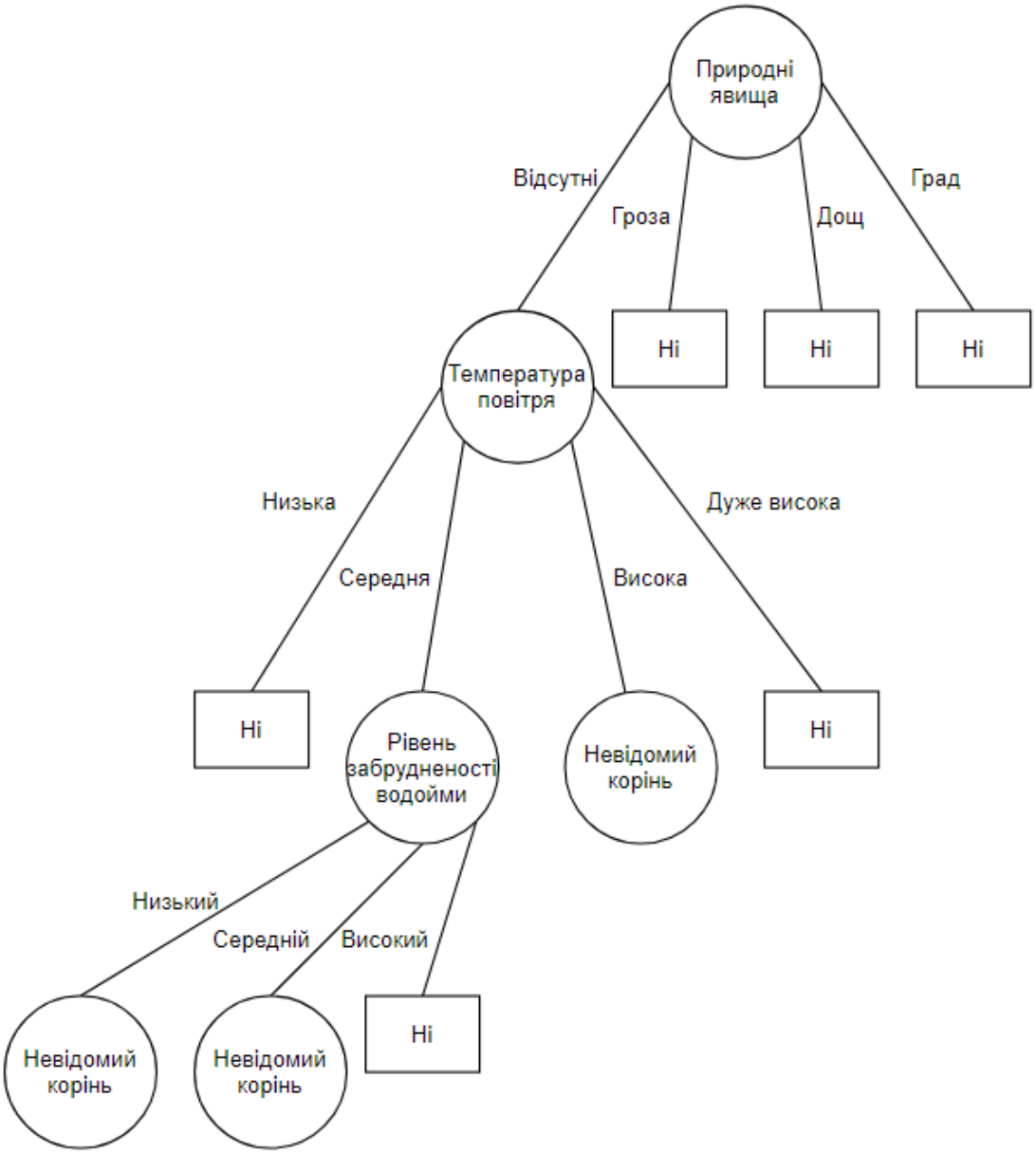


Рис. 4.7. Проміжний варіант дерева рішень

Отримаємо нову підмножину

Таблиця 4.9

№ людини	*Стан здоров'я людини*	*Чи безпечно купатись*
2	*Незадовільний*	*Ні*
11	*Задовільний*	*Так*
16	*Задовільний*	*Так*

Так як залишився всього один атрибут у множині, то можна побудувати листки від поточного вузла.

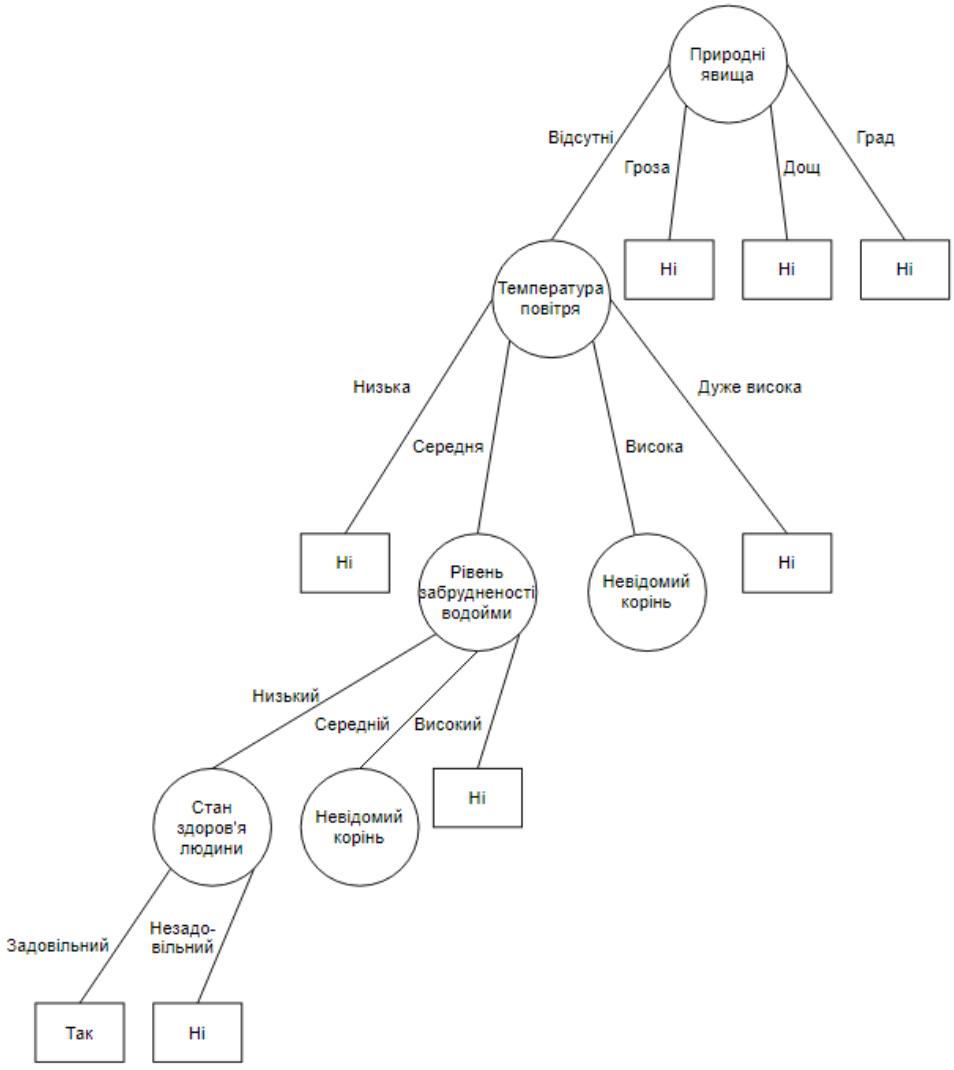


Рис. 4.8. Проміжний варіант дерева рішень

Так само будуємо й інші листки дерева

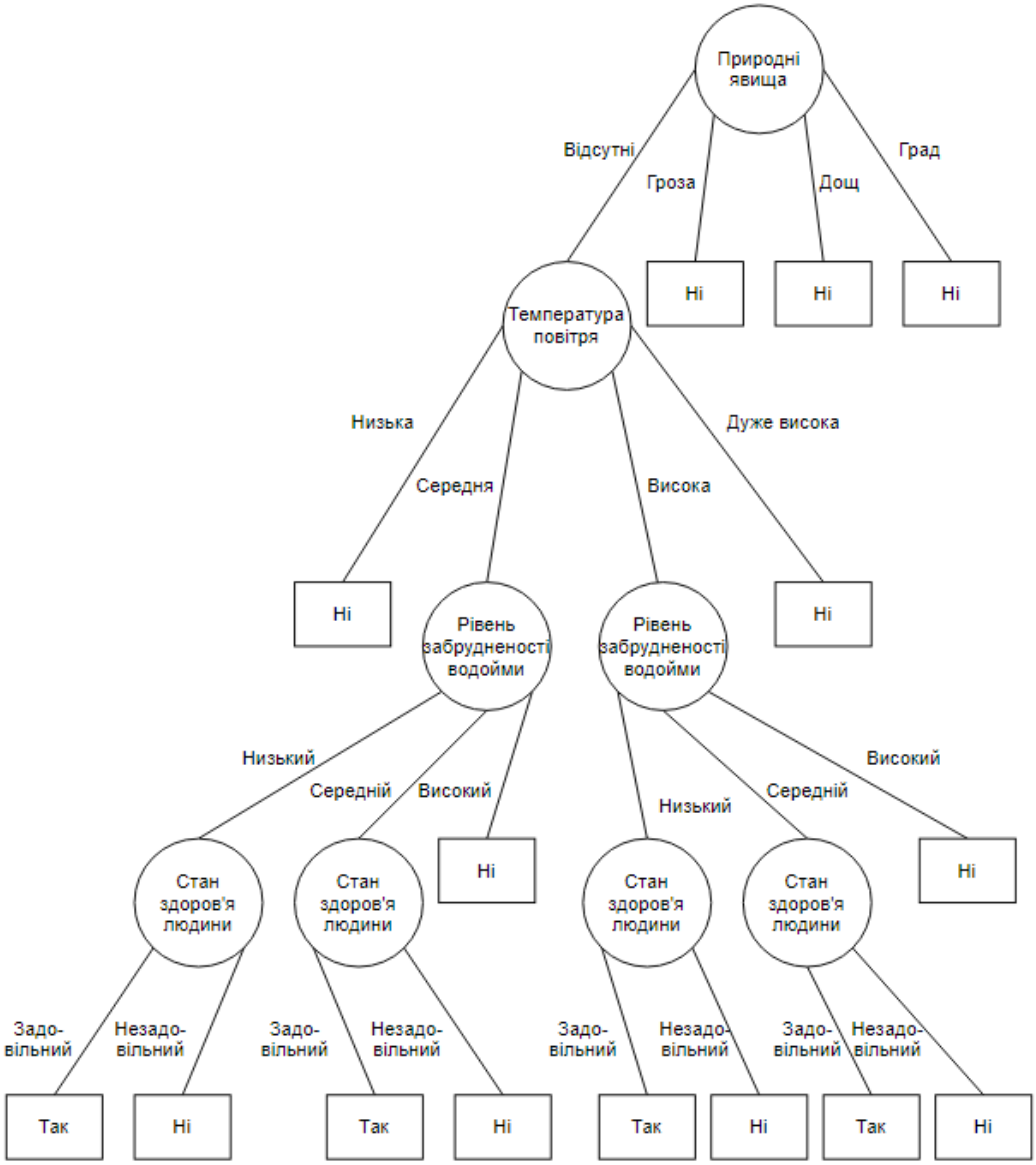


Рис. 4.9. Остаточний варіант дерева рішень

Виконання алгоритму завершено.

Алгоритм C4.5

C4.5 - алгоритм для побудови дерев рішень, розроблений Джоном Квінланом (англ. John Ross Quinlan). C4.5 є вдосконаленою версією алгоритму ID3 того ж автора. Зокрема, в нову версію було додано відсікання гілок (англ. pruning), можливість роботи з числовими атрибутами, а також можливість побудови дерева з неповної навчальної вибірки, в якій відсутні значення деяких атрибутів.

Для того, щоб за допомогою C4.5 побудувати вирішальне дерево і застосовувати його, дані повинні задовільняти декільком умовам.

Інформація про об'єкти, які необхідно класифікувати, повинна бути представлена у вигляді кінцевого набору ознак (атрибутів), кожен з яких має дискретне або числове значення. Такий набір атрибутів назвемо прикладом. Для всіх прикладів кількість атрибутів і їх склад мають бути постійними.

Безліч класів, на які будуть розбиватися приклади, повинні мати кінцеве число елементів, а кожен приклад має однозначно ставитися до конкретного класу. Для випадків з нечіткою логікою, коли приклади належать до класу із певною ймовірністю, C4.5 непридатний.

У навчальній вибірці кількість прикладів повинна бути значно більша кількості класів, до того ж кожен приклад має бути заздалегідь асоційований зі своїм класом. З цієї причини C4.5 є варіантом машинного навчання з учителем.

Нехай T є - навчальна вибірка прикладів, а C - множина класів, що складається з k елементів. Для кожного прикладу з T відома його приналежність до якогось із класів $C_1 \dots C_k$.

Побудова дерева рішень алгоритмом C4.5 принципово не відрізняється від його побудови в ID3. На першому кроці є корінь і асоційована з ним множина T, яку необхідно розбити на підмножини. Для цього необхідно обрати один з атрибутів в якості перевірки. Обраний атрибут A має n значень, що дає розбиття на підмножини. Далі створюються n нащадків кореня, кожному з яких поставлена у відповідність своя підмножина, отримана при розбитті T. Процедура вибору атрибута і розбиття по ньому рекурсивно застосовується до всіх n нащадків і зупиняється в двох випадках:

1) після чергового розгалуження в вершині опиняються приклади з одного класу (тоді вона стає листом, а клас, якому належать її приклади, буде рішенням),

2) вершина виявилася асоційованою з порожньою множиною (тоді вона стає листом, а в якості рішення обирається клас, який найбільш часто зустрічається у безпосередньому предку цієї вершини).

Ідею алгоритму можна представити у графічному вигляді (рис 2). За наявності в об’єктів лише двох змінних, їх можна представити у вигляді точок двовимірного простору. Об’єкти різних класів відмічені знаками «+» і «-». З рисунку видно, що при розбитті множини на підмножини будується дерево, що покриває тільки об’єкти вибраного класу.

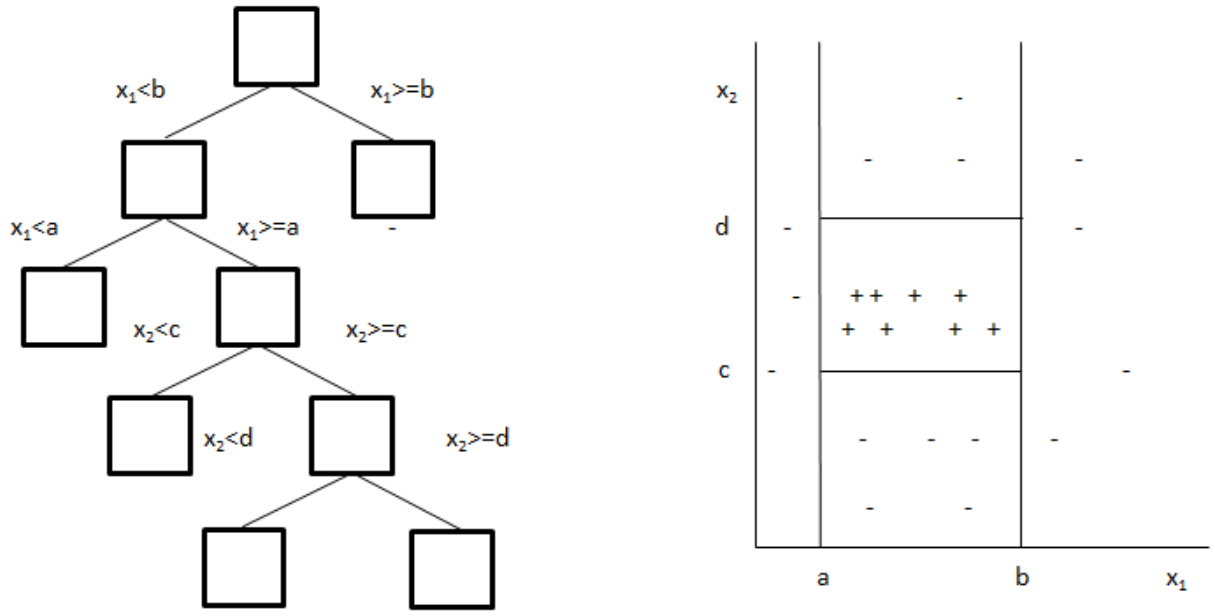


Рис. 4.10. Ідея алгоритму

Переваги алгоритму

- Працює з неперервними значеннями характеристик: на кожному кроці побудови дерева вибирає правило розбиття по одному з атрибутів. Розбиває об'єкти на кілька частин за цим правилом, рекурсивно запускається з отриманих підмножин;
- використовує редукцію для уникнення перенавчання;
- обробляє пропущені або аномальні значення ознак.

Недоліки алгоритму

- Виходять дуже гіллясті дерева через велику кількість унікальних значень атрибутів вибірки.

3. Приклад роботи алгоритму

Потрібно визначити чи відбудеться гра в м’яч, зважаючи на погодні умови. Відомі дані за минулі 14 днів зі всіма погодними умовами і з рішенням чи відбулась гра.

Таблиця 4.10

День	Прогноз	Температура	Вологість	Вітер	Грати в м’яч
D1	Сонячний	Гаряча	Висока	Слабкі	Ні
D2	Сонячний	Гаряча	Висока	Сильні	Ні
D3	Похмуро	Гаряча	Висока	Слабкі	Так
D4	Дощ	М'яка	Висока	Слабкі	Так
D5	Дощ	Прохолодний	Нормальний	Слабкий	Так
D6	Дощ	Прохолодний	Нормальний	сильний	Ні
D7	Похмуро	Прохолодний	Нормальний	Сильні	Так
D8	Сонячний	М'яка	Висока	Слабкі	Ні
D9	Сонячний	Прохолодний	Нормальний	Слабкий	Так
D10	Дощ	М'який	Нормальний	Слабкий	Так
D11	Сонячний	М'яка	Нормальний	Сильні	Так
D12	Похмуро	М'яка	Висока	Сильні	Так
D13	Похмуро	Гарячі	Нормальний	Слабкий	Так
D14	Дощ	М'яка	Висока	Сильні	Ні

Прогноз

Таблиця 4.11

День	Прогноз	Грати в м’яч
D1	Сонячний	Ні
D2	Сонячний	Ні
D3	Похмуро	Так
D4	Дощ	Так
D5	Дощ	Так
D6	Дощ	Ні
D7	Похмуро	Так
D8	Сонячний	Ні
D9	Сонячний	Так
D10	Дощ	Так
D11	Сонячний	Так
D12	Похмуро	Так
D13	Похмуро	Так
D14	Дощ	Ні

GAIN(РІШЕННЯ, ПРОГНОЗ) = ENTROPY(Р)- ∑(P(P|П).ENTROPY(P|П))

GAIN(РІШЕННЯ, ПРОГНОЗ) = ENTROPY(Р) – P(P|П-СОНЯЧНИЙ). ENTROPY(P|П=СОНЯЧНИЙ) – P(P|П=ПОХМУРО). ENTROPY(P|П=ПОХМУРО) – P(P|П=ДОЩ). ENTROPY(P|П=ДОЩ)

Є 5 РІШЕНЬ ІЗ ЗНАЧЕННЯМ СОНЯЧНИЙ (3 – НІ, 2 – ТАК), 4 ЗІ ЗНАЧЕННЯМ ПОХМУРО (0 – НІ, 4 – ТАК), 5 ЗІ ЗНАЧЕННЯМ ДОЩ (2 – НІ, 3 – ТАК).

ENTROPY(P|П=СОНЯЧНИЙ) = -(3/5).LOG2(3/5) – (2/5).LOG2(2/5) = 0.442 + 0.529 = 0.971

ENTROPY(P|П=ПОХМУРО) = -(0/4).LOG2(0/4) – (4/4).LOG2(4/4) = 0

ENTROPY(P|П=ДОЩ) = -(2/5).LOG2(2/5) – (3/5).LOG2(3/5) = 0.529 + 0.442 = 0.971

GAIN(P, П) = -(5/14).LOG2(5/14) – (9.14).LOG2(9/14) – (5/14).(0.971) – (4/14).(0) – (5/14).(0.971) = 0.940 – 0.347 – 0 – 0.347 = 0.246

SPLITINFO(P, П) = -(5/14).LOG(5/14) – (4/14).LOG2(4/14) – (5/14).LOG(5/14) = 0.53 + 0.516 + 0.53 = 1.576

GAINRATIO(РІШЕННЯ, ПРОГНОЗ) = GAIN(P, П)/ SPLITINFO(P, П) = 0.246/1.576 = 0.156

Температура

Таблиця 4.12

День	Температура	Грати в м'яч
D1	Гаряча	Ні
D2	Гаряча	Ні
D3	Гаряча	Так
D4	М'яка	Так
D5	Прохолодний	Так
D6	Прохолодний	Ні
D7	Прохолодний	Так
D8	М'яка	Ні
D9	Прохолодний	Так
D10	М'який	Так
D11	М'яка	Так
D12	М'яка	Так
D13	Гарячі	Так
D14	М'яка	Ні

GAIN(РІШЕННЯ, ТЕМПЕРАТУРА) = ENTROPY(P)- ∑(P(P|T).ENTROPY(P|T))
GAIN(РІШЕННЯ, ТЕМПЕРАТУРА) = ENTROPY(P) – P(P|T=ГАРЯЧА). ENTROPY(P|T=ГАРЯЧА) – P(P|T=М'ЯКА). ENTROPY(P|T=М'ЯКА) – P(P|T=ПРОХОЛОДНИЙ). ENTROPY(P|T=ПРОХОЛОДНИЙ)

Є 4 РІШЕНЬ ІЗ ЗНАЧЕННЯМ ГАРЯЧА(2 – НІ, 2 – ТАК), 4 ЗІ ЗНАЧЕННЯМ ПРОХОЛОДНИЙ(1 – НІ, 3 – ТАК), 6 ЗІ ЗНАЧЕННЯМ М'ЯКА (2 – НІ, 4 – ТАК).

ENTROPY(P|T=ГАРЯЧА) = -(2/4).LOG2(2/4) – (2/4).LOG2(2/4) = 1
ENTROPY(P|T=ПРОХОЛОДНИЙ) = -(1/4).LOG2(1/4) – (3/4).LOG2(3/4) = 0.811

ENTROPY(P|T=М'ЯКА) = -(2/6).LOG2(2/6) – (4/6).LOG2(4/6) = 0.918

GAIN(P, T) = -(5/14).LOG2(5/14) – (9.14)*LOG2(9/14) – (4/14).(1) – (4/14).(0.811) – (6/14).(0.918) = 0.940 – 0.286 – 0.23 – 0.39 = 0.034

SPLITINFO(P, T) = -(4/14).LOG2(4/14) – (4/14).LOG2(4/14) – (6/14).LOG2(6/14) = 0.516 + 0.516 + 0.524 = 1.556

GAINRATIO(РІШЕННЯ, ТЕМПЕРАТУРА) = GAIN(P, T)/ SPLITINFO(P, T) = 0.034/1.556 = 0.022

Вологість

Таблиця 4.13

День	Вологість	Грати в м'яч
D1	Висока	Ні
D2	Висока	Ні
D3	Висока	Так
D4	Висока	Так
D5	Нормальний	Так
D6	Нормальний	Ні
D7	Нормальний	Так
D8	Висока	Ні
D9	Нормальний	Так
D10	Нормальний	Так
D11	Нормальний	Так
D12	Висока	Так
D13	Нормальний	Так
D14	Висока	Ні

GAIN(РІШЕННЯ, ВОЛОГІСТЬ) = ENTROPY(P)- ∑(P(P|B).ENTROPY(P|B))
GAIN(РІШЕННЯ, ВОЛОГІСТЬ) = ENTROPY(P) – P(P|B=ВИСОКА). ENTROPY(P|B=ВИСОКА) – P(P|B=НОРМАЛЬНИЙ). ENTROPY(P|B=НОРМАЛЬНИЙ)

Є 7 РІШЕНЬ ІЗ ЗНАЧЕННЯМ ВИСОКА(4 – НІ, 3 – ТАК), 7 ЗІ ЗНАЧЕННЯМ НОРМАЛЬНИЙ(1 – НІ, 6 – ТАК)

ENTROPY(P|B=ВИСОКА) = -(4/7).LOG2(4/7) – (3/7).LOG2(3/7) = 0.985
ENTROPY(P|B=НОРМАЛЬНИЙ) = -(1/7).LOG2(1/7) – (6/7).LOG2(6/7) = 0.592

GAIN(P, B) = -(5/14).LOG2(5/14) – (9.14).LOG2(9/14) – (7/14).(0.985) – (7/14).(0.592) = 0.940 – 0.4925 – 0.296 = 0.1515

SPLITINFO(P, B) = -(7/14).LOG2(7/14) – (7/14).LOG2(7/14) = 1

GAINRATIO(РІШЕННЯ, ВОЛОГІСТЬ) = GAIN(P, B)/ SPLITINFO(P, B) = 0.1515/1 = 0.1515

Вітер

Таблиця 4.14

День	Вітер	Грати в м'яч
D1	Слабкі	Ні
D2	Сильні	Ні
D3	Слабкі	Так
D4	Слабкі	Так
D5	Слабкий	Так
D6	сильний	Ні
D7	Сильні	Так
D8	Слабкі	Ні
D9	Слабкий	Так
D10	Слабкий	Так
D11	Сильні	Так
D12	Сильні	Так
D13	Слабкий	Так
D14	Сильні	Ні

GAIN(РІШЕННЯ, ВІТЕР) = ENTROPY(P)- ∑(P(P|B).ENTROPY(P|B))
GAIN(РІШЕННЯ, ВІТЕР) = ENTROPY(P) – P(P|B=СЛАБКИЙ). ENTROPY(P|B=СЛАБКИЙ) – P(P|B=СИЛЬНИЙ). ENTROPY(P|B=С ИЛЬНИЙ)

Є 8 РІШЕНЬ ІЗ ЗНАЧЕННЯМ СЛАБКИЙ(2 – НІ, 6 – ТАК), 6 ЗІ ЗНАЧЕННЯМ СИЛЬНИЙ(3 – НІ, 3 – ТАК)

ENTROPY(P|B=СЛАБКИЙ) = -(2/8).LOG2(2/8) – (6/8).LOG2(6/8) = 0.811
ENTROPY(P|B=СИЛЬНИЙ) = -(3/6).LOG2(3/6) – (3/6).LOG2(3/6) = 1

GAIN(P, B) = -(5/14).LOG2(5/14) – (9.14).LOG2(9/14) – (8/14).(0.811) – (6/14).(1) = 0.940 – 0.463 – 0.429 = 0.048

SPLITINFO(P, B) = -(8/14).LOG2(8/14) – (6/14).LOG2(6/14) = 0.985

GAINRATIO(РІШЕННЯ, ВОЛОГІСТЬ) = GAIN(P, B)/ SPLITINFO(P, B) = 0.048/0.985 = 0.049

Результати

Таблиця 4.15

АТРИБУТ	**GAIN**	**GAINRATIO**
ПРОГНОЗ	0.246	0.156
ТЕМПЕРАТУРА	0.034	0.022
ВОЛОГІСТЬ	0.151	0.151
ВІТЕР	0.048	0.049

Розбиття

Таблиця 4.16

День	Прогноз	Температура	Вологість	Вітер	Грати в м’яч
D1	Сонячний	Гаряча	Висока	Слабкий	Ні
D2	Сонячний	Гаряча	Висока	Сильний	Ні
D8	Сонячний	М’яка	Висока	Слабкий	Ні
D9	Сонячний	Прохолодна	Нормальна	Слабкий	Так
D11	Сонячний	М’яка	Нормальна	Сильний	Так

Таблиця 4.17

День	*Прогноз*	*Температура*	*Вологість*	*Вітер*	*Грати в м’яч*
D3	*Похмуро*	*Гаряча*	*Висока*	*Слабкий*	*Так*
D7	*Похмуро*	*Прохолодна*	*Нормальна*	*Сильний*	*Так*
D12	*Похмуро*	*М’яка*	*Висока*	*Сильний*	*Так*
D13	*Похмуро*	*Гаряча*	*Нормальна*	*Слабкий*	*Так*

Таблиця 4.18

День	*Прогноз*	*Температура*	*Вологість*	*Вітер*	*Грати в м’яч*
D4	*Дощ*	*М’яка*	*Висока*	*Слабкий*	*Так*
D5	*Дощ*	*Прохолодна*	*Нормальна*	*Слабкий*	*Так*
D6	*Дощ*	*Прохолодна*	*Нормальна*	*Сильний*	*Ні*
D10	*Дощ*	*М’яка*	*Нормальна*	*Слабкий*	*Так*
D14	*Дощ*	*М’яка*	*Висока*	*Сильний*	*Ні*

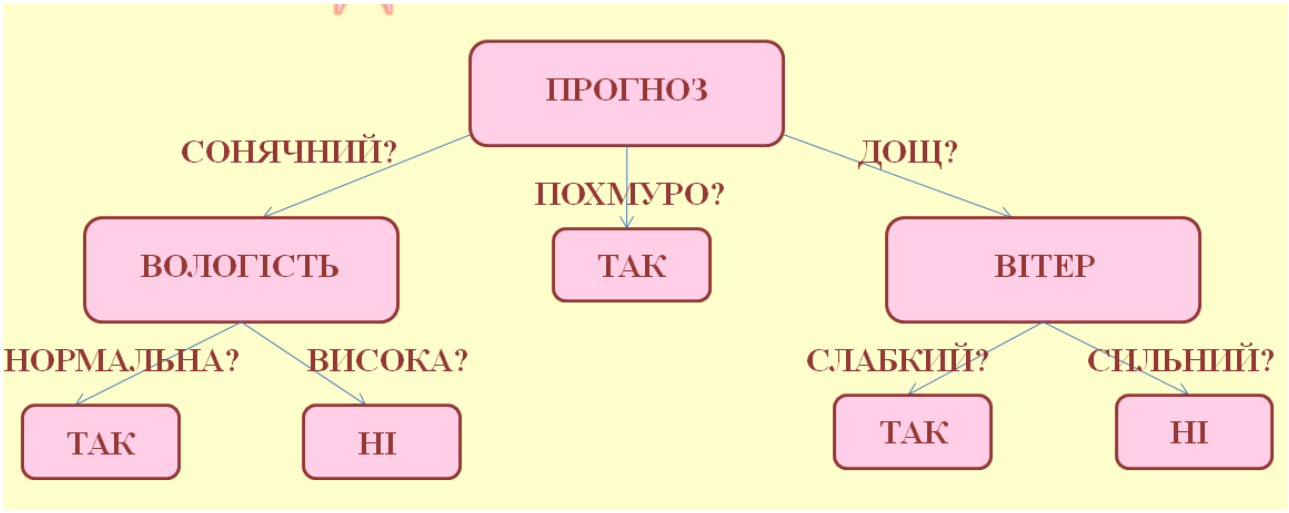


Рис. 4.11. Остаточний варіант дерева рішень

Переваги використання «дерев рішень» та області їх застосування

Розглянувши основні проблеми, що виникають при побудові дерев, варто також перелічити деякі їх переваги:

- швидкий процес навчання;
- генерація правил в областях, де експертові важко формалізувати свої знання;
- формулювання правил природною мовою;
- інтуїтивно зрозуміла класифікаційна модель;
- висока точність прогнозування в порівнянні з іншими методами (статистика, нейронні мережі);

В силу цих та багатьох інших причин, методологія дерев рішень є важливим інструментом у роботі кожного фахівця, що займається аналізом даних. Дерева рішень є зручним інструментом у системах підтримки прийняття рішень, інтелектуального аналізу даних (data mining), особливо в галузях, де високою є ціна помилки.

Дерева рішень успішно використовуються для вирішення практичних завдань у таких областях:

- Банківська справа (оцінка кредитоспроможності клієнтів банку при видачі кредитів).
- Промисловість (контроль за якістю продукції (виявлення дефектів), випробування без руйнувань, наприклад, перевірка якості зварювання, тощо).
- Медицина (діагностика різних захворювань).
- Молекулярна біологія (аналіз будови амінокислот).